10Pearls AQI Predictor

Technical Project Documentation

Serverless End-to-End Air Quality Index Forecasting System for Lahore, Pakistan

Author: Muhammad Inam Shahid
Program: 10Pearls Internship Program
Repository: github.com/inaam78/10pearls-AQI-Predictor

Table of Contents

1. Executive Summary
2. Project Objectives
3. System Architecture
4. Technology Stack & Alternative Tool Choices
5. Feature Pipeline & Engineering
6. Historical Backfill
7. Model Training & Evaluation
8. Automation & CI/CD
9. Web Dashboard (app.py)
10. AQI Reference Scale
11. Additional Guidelines: Status & Gaps
12. Repository Structure
13. Setup & Installation
14. Known Limitations & Recommendations
15. Conclusion

1. Executive Summary

The 10Pearls AQI Predictor is an end-to-end, serverless machine learning system that forecasts Air Quality Index (AQI) and PM2.5 concentration levels for Lahore, Pakistan, up to 72 hours in advance. The system was developed as a practical capstone project during the 10Pearls Internship Program, and implements the complete machine learning lifecycle: automated feature engineering, a versioned feature store, model training and evaluation, and an interactive forecasting dashboard.

This document provides complete technical documentation of the system as implemented — its architecture, the actual tools and libraries used (including deliberate substitutions made in place of the tools originally suggested in the project brief), the feature and model design, the dashboard functionality, and an honest account of current limitations and recommended next steps.

2. Project Objectives

As defined in the original project brief, the system was required to:

- Fetch raw weather and pollutant data from an external API.
- Compute model input features and prediction targets, including time-based features (hour, day, month) and derived features such as AQI change rate.
- Store engineered features in a feature store for reuse between training and inference.
- Backfill historical features and targets to build a training dataset.
- Train and evaluate machine learning models (e.g. Random Forest, Ridge Regression, or deep learning alternatives), scored on RMSE, MAE, and R^2.
- Store the trained model in a model registry.
- Automate the feature and training pipelines to run on a schedule (feature pipeline hourly, training pipeline daily) via a CI/CD tool.
- Serve predictions through an interactive web dashboard (Streamlit/Gradio or Flask/FastAPI).
- Additionally: perform exploratory data analysis (EDA), use SHAP/LIME for feature importance, and add alerts for hazardous AQI levels.

3. System Architecture

The system follows a four-stage serverless MLOps pipeline: (1) a Feature Pipeline that ingests raw weather and pollution data and computes model-ready features, (2) a Feature Store that persists and versions those features, (3) a Training Pipeline that consumes stored features to train and evaluate a regression model, and (4) a Streamlit Dashboard that loads the trained model and the latest features to render forecasts and health advisories.

3.1 Architecture Diagram

Weather & Pollution API → Feature Pipeline → Feature Store (Feast) → Model Training → Model Registry → Streamlit Dashboard → End User

3.2 Data Flow Summary

- Raw data is retrieved from Open-Meteo's Weather and Air Quality APIs.
- The feature pipeline computes weather, pollutant, and time-based features, and writes them to a Parquet file consumed by the feature store.
- The Feast feature store registers and serves these features (offline for training, online-capable for low-latency serving).
- The training pipeline retrieves historical features, trains a Random Forest regression model, evaluates it (MAE, RMSE, R^2), and serializes the model artifact.
- The Streamlit dashboard loads the model and the latest 72-hour predictions, converts PM2.5 to AQI using EPA breakpoints, and renders forecasts, health advisories, and model diagnostics.

4. Technology Stack & Alternative Tool Choices

The original project brief suggested specific example tools for each stage of the pipeline, while explicitly noting that alternatives could and should be explored. The table below documents which tool was actually used at each stage, the brief's suggested alternative, and the rationale for the substitution.

Pipeline Stage	Brief's Suggested Tool	Tool Actually Used	Rationale for Substitution
Raw data source	AQICN or OpenWeather	Open-Meteo (Weather API + Air Quality API)	Open-Meteo offers a free, keyless, high-rate-limit API with both historical and forecast endpoints for weather and pollution data, avoiding the API-key provisioning and stricter rate limits of AQICN/OpenWeather — a better fit for a fully serverless, zero-cost pipeline.
Feature store	Hopsworks or Vertex AI	Feast (open-source feature store)	Feast is self-hostable and free with no account/quota constraints, integrates natively with Python and Parquet-based offline storage, and avoids vendor lock-in to a managed platform's free-tier limits — while still providing the core feature store abstractions (entities, feature views, TTL, online/offline serving) the brief calls for.
Model registry	(implied: a managed registry alongside Hopsworks/Vertex AI)	Local/versioned model artifact storage (joblib-serialized model file)	Given the serverless, cost-free constraint, model artifacts are serialized directly (e.g. via joblib) and referenced by the dashboard, rather than requiring a separate managed registry service.
Model training	scikit-learn (Random Forest, Ridge Regression) or TensorFlow/PyTorch	scikit-learn — Random Forest Regressor	Random Forest was selected as the primary model for its strong baseline performance on tabular, mixed-type environmental data without requiring the additional infrastructure or tuning overhead of a deep learning approach.
CI/CD automation	Apache Airflow or GitHub Actions	GitHub Actions	GitHub Actions runs natively alongside the repository with no separate infrastructure to provision or maintain, aligning with the 100%-serverless design goal.
Web application	Streamlit/Gradio or Flask/FastAPI	Streamlit	Streamlit provides the fastest path to an interactive, chart-rich dashboard directly in Python, without needing to hand-build a separate frontend as a Flask/FastAPI approach would require.

4.1 Full Technology Stack

Layer	Technology
Language	Python 3.11+
Data source	Open-Meteo Weather API & Air Quality API
Feature store	Feast (open-source feature store)

Layer	Technology
Data processing	pandas, NumPy
Machine learning	scikit-learn (Random Forest Regressor)
Model serialization	joblib
Visualization	Plotly, Matplotlib, Seaborn
Dashboard framework	Streamlit
Automation / CI/CD	GitHub Actions
Version control	Git / GitHub

5. Feature Pipeline & Engineering

The feature pipeline is responsible for transforming raw weather and pollution readings into a model-ready feature table. Feature definitions are declared centrally in feature_definitions.py using Feast's Python SDK, giving the project a single, versioned source of truth for what a "feature" means across training and inference.

5.1 Feast Feature Store Configuration

Component	Configuration
Feast project name	adequate_stud
Entity	aqi_location (join key: location_id)
Data source	FileSource — data/lahore_features.parquet
Timestamp field	event_timestamp
Feature view	lahore_air_quality_features
Time-to-live (TTL)	7 days
Online serving	Enabled
Tags	team: aqi_prediction · project: lahore_aqi · environment: development

5.2 Engineered Feature Set

Weather (meteorological) features:

- temperature_2m — air temperature at 2 metres
- relative_humidity_2m — relative humidity at 2 metres
- precipitation
- surface_pressure
- wind_speed_10m and wind_direction_10m — wind at 10 metres

Air quality (pollutant) features:

- pm10, pm2_5
- carbon_monoxide
- nitrogen_dioxide
- sulphur_dioxide
- ozone

- dust

Time-based features:

- hour, day, day_of_week, month, year
- is_weekend (binary flag)

The project brief additionally calls out a derived "AQI change rate" feature. This is a natural extension of the feature set above — a lag/diff of the AQI or PM2.5 series — and is recommended as a documented enhancement in the feature computation logic (see Section 14).

6. Historical Backfill

To build a training dataset, the feature-generation logic is run across a historical date range rather than only for the current moment. This produces a historical (features, targets) table sufficient to train a supervised regression model, following the same feature computation logic used for live inference — ensuring training/serving consistency, which is one of the primary benefits a feature store like Feast is designed to guarantee.

7. Model Training & Evaluation

The training pipeline retrieves historical features and targets from the feature store, trains a Random Forest Regressor to predict AQI/PM2.5 over a 72-hour forecast horizon, and evaluates the resulting model against three standard regression metrics.

7.1 Reported Model Performance

Metric	Value	Interpretation
MAE (Mean Absolute Error)	28.56 µg/m³	On average, forecasts deviate from actual PM2.5 by ~28.6 µg/m³.
RMSE (Root Mean Squared Error)	40.54 µg/m³	Penalizes larger errors more heavily than MAE; indicates some higher-magnitude misses.
R^2 (Coefficient of Determination)	0.4681	The model explains roughly 47% of the variance in PM2.5 — a reasonable baseline with clear room for improvement.

> **Note:** *An R^2 of 0.47 indicates a usable baseline model rather than a highly tuned one, with clear room for improvement at longer points in the 72-hour horizon (see Section 14, Recommendations).*

7.2 Model Selection Rationale

Random Forest Regressor was chosen as the primary model because it handles mixed-scale tabular features (temperature, humidity, pollutant concentrations, categorical time features) without requiring feature scaling, is robust to outliers common in raw sensor/API data, and provides built-in feature importance — useful both for interpretability and as a lightweight substitute for a full SHAP/LIME analysis where the latter has not yet been implemented.

8. Automation & CI/CD

The repository includes a GitHub Actions workflow configuration under .github/workflows/, in line with the project brief's requirement to automate the feature pipeline (suggested hourly) and the training pipeline (suggested daily) without relying on any always-on server infrastructure.

The exact trigger configuration and schedule (cron expressions, manual dispatch, or push triggers) should be verified against the current .github/workflows/ YAML file(s) and documented here precisely, since workflow configurations are commonly iterated on after initial documentation is written.

9. Web Dashboard (app.py)

The dashboard is implemented as a single-page Streamlit application (app.py) that loads the trained model and the latest generated predictions, then renders them across four functional tabs.

9.1 Dashboard Tabs

Tab	Purpose
72-Hour Forecast	Displays the AQI/PM2.5 forecast in hourly or Day 1/2/3 daily-summary view, with optional AQI severity band overlays and confidence bounds.
Health & Action Advisories	Surfaces outdoor-activity and indoor-precaution guidance that adapts automatically to the current predicted AQI level.
Model Performance	Displays MAE, RMSE, and R^2 for the current model, plus a feature-importance visualization.
Raw Data & Export	Allows the user to inspect and download the full 72-hour forecast as a CSV file.

9.2 Sidebar Controls

- City selector (currently Lahore only)
- Forecast view mode toggle: Hourly vs. Daily
- AQI severity band overlay toggle
- Prediction confidence bounds toggle

9.3 PM2.5-to-AQI Conversion

The dashboard converts the model's raw PM2.5 output into a US EPA-standard AQI value using the official EPA breakpoint formula, ensuring the displayed AQI figure is directly comparable to publicly reported AQI values rather than being a bespoke internal scale.

10. AQI Reference Scale

AQI Range	Category	PM2.5 (µg/m³)
0–50	Good	0.0 – 12.0
51–100	Moderate	12.1 – 35.4
101–150	Unhealthy for Sensitive Groups	35.5 – 55.4
151–200	Unhealthy	55.5 – 150.4
201–300	Very Unhealthy	150.5 – 250.4
301+	Hazardous	250.5+

11. Additional Guidelines: Status & Gaps

The project brief's supplementary guidelines, and their current implementation status, are summarized honestly below.

Guideline	Status
Perform EDA to identify seasonal/pollution trends	Recommended as an explicit, documented notebook/report deliverable if not already present — see Recommendations.

Guideline	Status
Use a variety of forecasting models (statistical → deep learning)	Random Forest is implemented as the primary model. Benchmarking against Ridge Regression and/or a deep learning model (e.g. LSTM/PyTorch) is a documented next step.
Use SHAP or LIME for feature importance	Random Forest's built-in feature importance is exposed in the dashboard as an interim solution. A dedicated SHAP/LIME analysis is recommended for deeper, per-prediction explainability.
Add alerts for hazardous AQI levels	The dashboard's Health & Action Advisories tab surfaces category-based guidance; explicit push/email/SMS-style hazardous-level alerting is a recommended enhancement.

12. Repository Structure

Path	Description
.github/workflows/	GitHub Actions workflow definitions for pipeline automation.
assets/	Static assets used by the project/dashboard.
data/	Raw and/or processed data files, including the Parquet source consumed by Feast.
feature_store/adequate_stud/	Feast feature repository (project auto-named "adequate_stud" by Feast's initialization).
results/predictions/	Generated forecast output, e.g. the latest 72-hour prediction CSV.
results/models/	Model evaluation output, e.g. test performance metrics CSV.
src/	Data fetching, feature engineering, and model training scripts.
app.py	Streamlit dashboard application entry point.
feature_definitions.py	Central Feast feature/entity/feature-view definitions.
requirements.txt	Python dependency list.

13. Setup & Installation

13.1 Prerequisites

- Python 3.11 or later
- Git

13.2 Installation Steps

- Clone the repository: git clone https://github.com/inaam78/10pearls-AQI-Predictor.git
- Create and activate a virtual environment (recommended).
- Install dependencies: pip install -r requirements.txt
- Run the feature and training pipeline scripts under src/ to populate the feature store and model artifacts.
- Launch the dashboard: streamlit run app.py

requirements.txt now pins exact versions for all 24 listed packages (e.g. pandas==3.0.5, scikit-learn==1.9.0) — a genuine reproducibility improvement over an unpinned file. However, streamlit, plotly, and feast are still not listed, despite being imported directly by app.py and feature_definitions.py; installing strictly from requirements.txt will still raise an ImportError until these three are added with matching version pins.

13.3 Current requirements.txt Contents (Verified)

Package	Pinned Version
certifi	2026.7.22
charset-normalizer	3.4.9
contourpy	1.3.3
cycler	0.12.1
fonttools	4.63.0
idna	3.18
joblib	1.5.3
kiwisolver	1.5.0
matplotlib	3.11.1
narwhals	2.24.0
numpy	2.4.6
packaging	26.3
pandas	3.0.5
pillow	12.3.0
pyparsing	3.3.2
python-dateutil	2.9.0.post0
requests	2.34.2
scikit-learn	1.9.0
scipy	1.17.1
seaborn	0.13.2
six	1.17.0
threadpoolctl	3.6.0
tzdata	2026.3
urllib3	2.7.0

14. Known Limitations & Recommendations

14.1 Known Limitations

- Model performance ($R^2 \approx 0.47$) indicates meaningful forecast error remains, particularly at longer horizons within the 72-hour window.
- The system currently forecasts for a single city (Lahore) only.
- SHAP/LIME-based explainability and hazardous-level alerting are not yet fully implemented per the brief's guidelines.
- No LICENSE file is currently included in the repository.

14.2 Recommendations

- Benchmark Random Forest against Ridge Regression and a deep learning baseline (e.g. LSTM) to validate model choice against the brief's "best possible model" requirement.
- Add a dedicated SHAP analysis notebook/module for per-feature and per-prediction explainability.
- Implement explicit hazardous-AQI alerting (e.g. email/webhook) beyond the current in-dashboard advisory text.
- Document the exact GitHub Actions schedule and add status badges to the README for pipeline health visibility.
- Add a LICENSE file appropriate to the project's intended reuse terms.

15. Conclusion

The 10Pearls AQI Predictor successfully implements the core architecture required by the project brief — a feature pipeline, a versioned feature store, a trained and evaluated regression model, and an interactive forecasting dashboard — while substituting several suggested tools (Feast for Hopsworks/Vertex AI, Open-Meteo for AQICN/OpenWeather) with justified, cost-free, serverless-compatible alternatives. The system provides a working, end-to-end demonstration of an MLOps workflow applied to a real, socially relevant forecasting problem, with clearly identified next steps toward a more complete and production-hardened implementation.
